CNN – II

Structured Notes

Convolution as a Linear Operation,

Dot Product Formulation,
Spatial Information Preservation, and Linear Transformations

Contents

1 Recap: Why CNN Over ANN for Images **2**

2 Convolution as a Linear Operation **2**
2.1 Convolution at One Location 2
2.2 Visual vs. Mathematical Interpretation 2
2.3 Worked Example: 2×2 Kernel 3
2.4 Coding the Convolution: Dot Product 3

3 Why CNN Does Not Lose Spatial Information **3**

4 Reshaping Matrices: The `reshape(-1)` Operation **3**

5 PyTorch Verification: Dot Product Equals Element-wise Convolution **4**

6 Convolution as a Linear Transformation **4**

7 Summary **5**

1 Recap: Why CNN Over ANN for Images

Problem with ANN for Images

An **Artificial Neural Network (ANN)** requires flattening the entire image into a single vector before processing. This flattening operation destroys the **spatial information** — the relative arrangement of pixels, edges, and regions — making ANN fundamentally unsuitable for image data.

Key points:

- ANN flattens the *complete* image into a 1D vector, losing all spatial relationships between pixels.
- This loss of spatial information is why ANN does not work well for image tasks.
- CNN was introduced specifically to address this limitation — it processes images in *patches* rather than flattening the entire image, thereby preserving spatial structure.

Why CNN Preserves Spatial Information

Unlike ANN, CNN does **not** flatten the entire image. Instead, it flattens only small **patches** of the image at a time. The outputs from all patches are stored in a **2D tensor** (grid shape), preserving the spatial arrangement of the original image.

2 Convolution as a Linear Operation

2.1 Convolution at One Location

At each location in the image, convolution performs the following operation:

1. Extract a **patch** from the image (same size as the kernel/filter).
2. **Multiply** each element of the patch with the corresponding element of the kernel.
3. **Sum** all the products to produce a single output value.

This is fundamentally a **dot product** operation.

Convolution at one location (3×3 example):

Given an image patch of size 3×3 and a kernel of size 3×3:

1. Flatten both the patch and the kernel into vectors of size 9 (i.e., 9×1).
2. Compute the output as:

$$\text{output} = \mathbf{w}^\top \mathbf{x}$$

where $\mathbf{w}$ is the flattened kernel and $\mathbf{x}$ is the flattened patch.

This is a **single convolution output** — one scalar value from one patch position.

2.2 Visual vs. Mathematical Interpretation

Visual interpretation: A 3×3 kernel slides over a 3×3 patch. Each element of the kernel multiplies the corresponding element of the patch, and the results are summed.

Mathematical interpretation: Both the patch and kernel are flattened into column vectors, and their dot product is computed:

$$\text{output} = \mathbf{w}^\top \mathbf{x} = \sum_{i=1}^{9} w_i \cdot x_i$$

Both approaches yield **exactly the same result**. The flattening and dot product formulation is how the operation is implemented in code.

Order of Flattening

When flattening both the patch and the kernel, the same ordering must be used for both (either row-wise or column-wise). The specific ordering does not matter as long as the **indices are matched** — i.e., both are flattened in the same manner so that corresponding elements are aligned.

2.3 Worked Example: 2×2 Kernel

Consider a 2×2 kernel applied to a 2×2 patch of an image:

1. The kernel slides to the first position and covers a 2×2 region.
2. Each element is multiplied with the corresponding kernel weight, and results are summed to produce a single output value.
3. The kernel slides to the next position and the process repeats.
4. The output is stored in a grid (smaller than the input), preserving spatial structure.

No spatial structure is lost because new features are created from the old ones **in the same spatial arrangement**.

2.4 Coding the Convolution: Dot Product

In code, the convolution at one location is implemented as:

1. Flatten the patch: $2 \times 2 \to 4 \times 1$ (column vector).
2. Flatten the kernel: $2 \times 2 \to 4 \times 1$ (column vector).
3. Compute $\mathbf{w}^\top \mathbf{x}$ — this produces the same result as element-wise multiplication followed by summation.

3 Why CNN Does Not Lose Spatial Information

Patch-wise Flattening vs. Full Flattening

- **ANN:** Flattens the *complete* image into a single 1D vector → spatial information is destroyed.
- **CNN:** Flattens only a small *patch* at a time, computes the dot product with the kernel, and stores the result in a **2D output tensor** → spatial information is preserved.

A common question is: "If CNN also flattens data, how is spatial information preserved?"

The answer: CNN is **not** flattening the complete image. It is flattening **individual patches** one at a time:

- Each patch is flattened and multiplied with the kernel (dot product).
- After computing the result for one patch, the kernel moves to the next patch (**sliding window**).
- The results are stored in a grid structure, creating a smaller but still **2D tensor**.
- This structured approach is the "same concept as ANN, but in a very structured manner" so that no spatial information is lost.

After one full pass of the sliding window, the output is a 2D feature map (not a 1D vector), which retains the spatial relationships from the original image.

4 Reshaping Matrices: The `reshape(-1)` Operation

To convert a 2D matrix into a single column vector:

$$3 \times 3 \xrightarrow{\text{reshape}(-1)} 9 \times 1$$

$$2 \times 2 \xrightarrow{\text{reshape}(-1)} 4 \times 1$$

The `-1` argument tells NumPy/PyTorch to automatically compute the required size, effectively flattening the matrix into a column vector.

Key points about reshaping:

- `reshape(-1)` converts any 2D matrix into a single column vector.
- Alternatively, `.flatten()` achieves the same result.
- Any matrix can be reshaped as long as the total number of elements is preserved. For example, a 4×8 matrix (32 elements) can be reshaped to 16×2, 32×1, etc.
- This is the same reshaping concept from NumPy, where array shapes can be changed freely using `.reshape()`.

5 PyTorch Verification: Dot Product Equals Element-wise Convolution

The equivalence between element-wise multiplication with summation and the dot product formulation can be verified in PyTorch:

Verifying Convolution as Dot Product

```python
import torch

# 3x3 image patch and 3x3 kernel
patch = torch.tensor([[1, 2, 3],
                      [4, 5, 6],
                      [7, 8, 9]], dtype=torch.float32)

kernel = torch.tensor([[1, 0, -1],
                       [1, 0, -1],
                       [1, 0, -1]], dtype=torch.float32)

# Method 1: Element-wise multiply then sum
manual_result = (patch * kernel).sum()

# Method 2: Flatten both, then dot product
w = kernel.reshape(-1)    # 9x1
x = patch.reshape(-1)     # 9x1
dot_result = torch.dot(w, x)

# Both produce exactly the same result
print(manual_result)  # Same value
print(dot_result)     # Same value
```

Listing 1: Dot product vs. element-wise multiplication

Both methods — `(patch * kernel).sum()` and `torch.dot(w, x)` — produce exactly the same output, confirming that convolution at one location is mathematically equivalent to a dot product.

6 Convolution as a Linear Transformation

Linear Transformation

A transformation T is **linear** if it satisfies:

$$T(a\mathbf{x}_1 + b\mathbf{x}_2) = a \cdot T(\mathbf{x}_1) + b \cdot T(\mathbf{x}_2)$$

for all scalars a, b and all vectors $\mathbf{x}_1, \mathbf{x}_2$.

Convolution (without activation functions) satisfies this property, making it a **linear operation**. Since the convolution output is $\mathbf{w}^\top \mathbf{x}$, which is a dot product (a linear operation), the convolution itself is a linear transformation.

Connection to Linearity of Expectation:

The linearity property of convolution is analogous to the **linearity of expectation** from probability:

$$E(aX + bY) = a \cdot E(X) + b \cdot E(Y)$$

Both follow the same principle: a linear operation applied to a weighted combination equals the weighted combination of the operation applied individually.

Why Linearity Matters

Since convolution is a linear transformation, stacking only convolution layers without activation functions would collapse into a single linear operation. This is why **non-linear activation functions** (such as ReLU) are essential between convolution layers — they introduce the non-linearity needed for the network to learn complex patterns.

7 Summary

- **ANN vs. CNN:** ANN loses spatial information by flattening the entire image; CNN preserves spatial information by processing small patches via a sliding window.
- **Convolution as dot product:** At each location, convolution flattens the patch and kernel into vectors and computes $\mathbf{w}^\top \mathbf{x}$. This is equivalent to element-wise multiplication followed by summation.
- **Single convolution output:** One patch $\times$ one kernel = one scalar value. The sliding window repeats this across the entire image to build a 2D output feature map.
- **No spatial loss:** CNN only flattens patches (not the entire image), and stores outputs in a 2D grid, preserving spatial structure.
- **Reshape with `-1`:** Converts a 2D matrix to a column vector (e.g., $3 \times 3 \to 9 \times 1$). The `.flatten()` method achieves the same result.
- **Convolution is linear:** $T(a\mathbf{x}_1 + b\mathbf{x}_2) = aT(\mathbf{x}_1) + bT(\mathbf{x}_2)$, analogous to linearity of expectation. Non-linear activations are needed to learn complex patterns.